

TSX Trading Advisor

Technical Reference Manual

Architecture, Configuration, Data Flow, and AI Logic

Classification: Confidential | Environment: Production | Platform: Windows / Python 3

Table of Contents

1. System Architecture Overview
2. File Structure & Roles
3. Configuration Reference (config.json / .env)
4. Data Engine (market_analyzer.py)
5. Technical Indicator Reference
6. AI Prompt System & Trading Rules
7. Telegram Daemon (telegram_daemon.py)
8. Scheduled Cron Jobs & Automated Tasks
9. Trade Ledger (trades.csv)
10. Advice History System (advice_history.txt)
11. Dependency Reference (requirements.txt)
12. Security Hardening
13. Operational Runbook
14. Troubleshooting Guide

1. System Architecture Overview

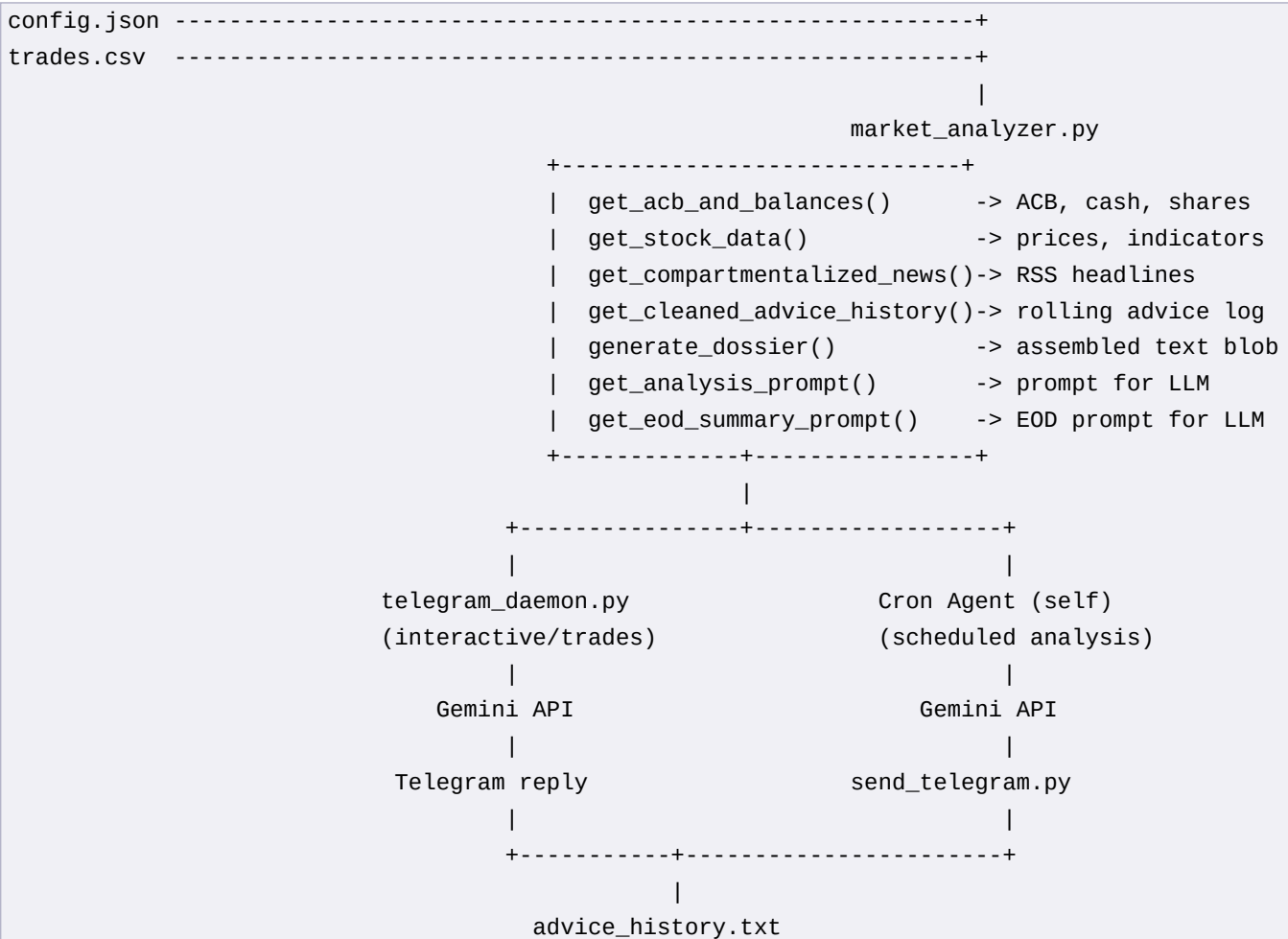
The TSX Trading Advisor is a locally-running, fully automated stock analysis and portfolio management system. It is built entirely on free, open-source Python libraries and requires no paid brokerage API. It connects to Yahoo Finance for real-time market data and uses the Google Gemini Large Language Model (LLM) to synthesize quantitative and qualitative signals into plain-language trading recommendations delivered via Telegram.

1.1 Core Execution Paths

The system runs along two independent, concurrent execution paths:

- **PATH A - Telegram Daemon (telegram_daemon.py):** A long-running foreground process started manually via `start_daemon.bat`. It listens for Telegram messages, handles natural-language trade logging, mode switching, on-demand analysis (`/analyze`), and general Q&A.
- **PATH B - Scheduled Cron Agent (Antigravity Agent):** An external AI agent (this same session) wakes up on a cron schedule (hourly, 3:30 PM, 4:05 PM ET on weekdays). It calls `market_analyzer.py` to build a live dossier, feeds it to Gemini, and sends the resulting recommendation via `send_telegram.py`.

1.2 Data Flow Diagram



1.3 Technology Stack

Component	Technology / Library	Version
Market Data	yfinance (Yahoo Finance)	1.4.1

Technical Indicators	pandas-ta	0.4.71b0
Data Frames	pandas	3.0.1
AI Reasoning	Google Gemini 2.5 Flash (via google-genai)	2.8.0
Telegram Messaging	python-telegram-bot	22.8
Secrets Management	python-dotenv	1.2.2
HTTP Requests	requests	latest
PDF Generation	fpdf2	2.8.7
Runtime	Python (Miniforge)	3.x

2. File Structure & Roles

File	Role
market_analyzer.py	Core data engine. Computes indicators, fetches news, builds dossier, and generates all LLM prompts.
telegram_daemon.py	Long-running bot server. Handles all user interaction: trade logging, analysis, Q&A, mode switching.
send_telegram.py	Standalone CLI utility. Sends a text string via Telegram Bot API. Used by the cron agent.
generate_pdf.py	Generates this PDF documentation from source.
config.json	Runtime configuration. Controls active mode (SHORT/MEDIUM) and portfolio-to-news-topic mapping.
trades.csv	Source of truth for all holdings. Records every trade; used to calculate ACB and cash balance.
advice_history.txt	Rolling log of AI-generated advice. Injected into every new dossier for context continuity.
.env	Secrets file. Contains TELEGRAM_BOT_TOKEN, TELEGRAM_CHAT_ID, GEMINI_API_KEY, and CHAT_
.gitignore	Excludes .env, trades.csv, and advice_history.txt from version control.
start_daemon.bat	Windows batch file to launch telegram_daemon.py from the correct working directory.
requirements.txt	Pinned Python dependency list for reproducible installs.
AGENT_RESTORE_INSTRUCTIO	Instructions for an AI agent to restore the system if the session is lost.
test_llm.py	Manual test script for validating the Gemini trade-extraction prompt.

3. Configuration Reference

3.1 config.json

This file controls the operating mode and portfolio composition. It is read at runtime by `market_analyzer.py` on every dossier generation. The user can modify the mode via Telegram by sending "Short Term Mode" or "Medium Term Mode".

```
{
  "mode": "SHORT",          // "SHORT" or "MEDIUM"
  "portfolio": {
    "CNQ.TO": ["CNQ.TO", "CL=F"], // Ticker -> list of RSS news topics
    "ABX.TO": ["ABX.TO", "GC=F"]  // Ticker -> list of RSS news topics
  }
}
```

- "mode": Controls which technical indicators are computed and which AI system instructions are used.
- "portfolio": Keys are TSX ticker symbols. Values are lists of Yahoo Finance RSS symbols whose headlines will be fetched as qualitative context for each holding.
- To add a new stock, add a new key-value pair to "portfolio". The system will automatically fetch data and news for it on the next run. No code changes required.

3.2 .env (Secrets)

All credentials are stored in a `.env` file in the project root. This file is excluded from version control by `.gitignore`. Never commit this file.

```
TELEGRAM_BOT_TOKEN=xxxxxxxx:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TELEGRAM_CHAT_ID=xxxxxxxx # Used by send_telegram.py to send messages
CHAT_ID=xxxxxxxx # Used by telegram_daemon.py to authorize incoming messages
GEMINI_API_KEY=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

- TELEGRAM_BOT_TOKEN: BotFather API token. Required by both the daemon and `send_telegram.py`.
- TELEGRAM_CHAT_ID: Your numeric Telegram chat ID. Required by `send_telegram.py`.
- CHAT_ID: Your numeric Telegram chat ID. Required by `telegram_daemon.py` for authorization. Any message from a different chat ID is silently dropped.
- GEMINI_API_KEY: Google AI Studio API key for the Gemini 2.5 Flash model.

3.3 Changing Operating Mode

Mode can be changed at runtime without restarting the daemon by sending a message to the Telegram bot.

```
User sends: "change mode"
Bot replies: "Please reply with Short Term Mode or Medium Term Mode"
User sends: "Short Term Mode"
Bot replies: "Operating mode successfully updated to: SHORT TERM"
```

4. Data Engine (market_analyzer.py)

market_analyzer.py is the central data and prompt generation engine. All other components import from it or call it as a subprocess. It has no side effects other than reading config.json, trades.csv, advice_history.txt, and (as a side-effect of get_cleaned_advice_history) rewriting advice_history.txt.

4.1 Function Reference

get_current_mode()

Reads config.json and returns "SHORT" or "MEDIUM". Falls back to "MEDIUM" if file is absent.

get_tickers()

Reads config.json and returns a list of ticker symbols from the portfolio keys. Falls back to ["CNQ.TO", "ABX.TO"] if the file is absent.

get_acb_and_balances()

Reads trades.csv row by row and computes two things: (1) Adjusted Cost Base (ACB) per ticker, computed from raw trade math (not from the CSV column). (2) Available Cash, read from the last Cash_Balance column value. Returns a dict:

```
{"positions": {"CNQ.TO": {"shares": 100.0, "total_cost": 6012.0, "acb": 60.12}}, "cash": 5230.00}
```

get_stock_data(ticker_symbol, mode)

Fetches 6 months of OHLCV history via yfinance (timeout=10s). Computes technical indicators via pandas-ta. Returns a dict of computed values depending on mode. Wraps everything in try/except; returns an error string if the ticker is invalid or Yahoo is unavailable.

get_compartmentalized_news()

Reads portfolio from config.json, builds a unique list of RSS topic symbols, then fetches the 10 most recent Yahoo Finance RSS headlines for each topic (timeout=10s). Organizes output by topic header. This is the Qualitative Analysis section of the dossier.

get_cleaned_advice_history()

IMPORTANT: This function has a write side-effect. It reads advice_history.txt, parses all entries by their [YYYY-MM-DD HH:MM:SS] timestamp headers, filters them, rewrites the file, and returns the cleaned content as a string for dossier injection. Retention policy: all entries from today, plus the final entry from each of the last 5 business days.

generate_dossier()

The main assembler. Calls all the above functions in order and concatenates their results into a single structured text block (the "dossier"). This dossier is the context fed to the Gemini LLM.

get_analysis_prompt(dossier=None, mode=None)

Wraps the dossier in the appropriate system instructions and output format rules for either SHORT or MEDIUM mode. Used by both the Telegram daemon (for /analyze and Q&A) and the cron agent (for scheduled hourly and 3:30 PM recommendations).

get_eod_summary_prompt(dossier=None)

Wraps the dossier in special end-of-day instructions that suppress active trading advice and instead request a narrative summary of portfolio performance and next-day outlook. Used exclusively by the 4:05 PM cron task.

5. Technical Indicator Reference

5.1 SHORT Mode Indicators (mode = "SHORT")

Designed for a 5-to-7 day momentum trading horizon. Optimized for mean-reversion signals.

Indicator	Parameters	Interpretation
EMA (Exponential Moving Average)	5-day, 9-day	Short-term momentum direction. EMA_5 crossing above
RSI (Relative Strength Index)	7-day	Overbought >70, Oversold <30. Faster than standard 14
ATR (Average True Range)	5-day	Measures raw volatility. Used to set stop-loss distances.
Bollinger Bands	20-day, 2 std devs	Price below BB_Lower = oversold/compressed. Price ab
Vol_Ratio	10-day rolling avg	Volume / 10-day average. >2.0 during a drop signals cap

5.2 MEDIUM Mode Indicators (mode = "MEDIUM")

Designed for a 2-to-4 week swing trading horizon. Emphasizes mean price and income events.

Indicator	Parameters	Interpretation
RSI (Relative Strength Index)	14-day	Standard 14-day RSI. Overbought >70, Oversold <30.
SMA (Simple Moving Average)	50-day	Key support/resistance level. Price above SMA_50 is str
Bollinger Bands (Upper)	50-day, 2 std devs	Profit-taking trigger. AI instructed to recommend trimmin
Ex-Dividend Date	from ticker.info	Upcoming ex-div dates affect entry timing. Holding befor

5.3 ACB (Adjusted Cost Base) Calculation

The system computes ACB from scratch on every run using a pure roll-forward method over trades.csv. It does NOT rely on the stored Shares_Balance or Cash_Balance columns for ACB computation (only cash reads from the column, for simplicity). The ACB formula:

```
On BUY or INIT:
    shares += quantity
    total_cost += quantity * price
    acb = total_cost / shares

On SELL:
    total_cost -= quantity * acb # Reduce cost at current ACB, not sell price
    shares -= quantity
    # If shares drop to 0, reset total_cost and acb to 0.0
```

6. AI Prompt System & Trading Rules

The AI prompt system is the decision-making core of the advisor. The system instructions embedded in the prompts constitute a formal rule set that the Gemini model must follow. These rules are different depending on the active mode.

6.1 Dossier Structure (Input to LLM)

The dossier is a structured plain-text document assembled by `generate_dossier()`. It contains:

- Section 1: Current Mode header (SHORT or MEDIUM)
- Section 2: Date and timestamp
- Section 3: Portfolio & Pricing -- shares owned, ACB, current price, all technical indicators per ticker
- Section 4: Qualitative Data -- 10 recent Yahoo Finance RSS headlines per configured news topic
- Section 5: Previous Advice History -- all of today's prior recommendations, plus the final advice from each of the last 5 business days

6.2 SHORT Mode System Instructions (all 9 rules)

Rule #	Name	Description
1	Mean Reversion	Price at or below BB_Lower = mathematically stretched.
2	Volume Shock / Capitulation	Vol_Ratio > 2.0 during a price drop = panic selling exhaust
3	Market Open Volatility	Before 9:30 AM ET, do NOT recommend buying. Wait 30
4	Finalized Macro Events	If news confirms a deal is done, anticipate the asset mov
5	Cash Constraint	If free cash < cost of purchase, BUY is strictly forbidden.
6	Position Sizing (Tranching)	Recommend buying in tranches (e.g., 25% of free cash)
7	Invalidation / Stop Loss	Every BUY recommendation must include a strict technic
8	Profit-Taking Mechanics	Price at BB_Upper AND RSI > 70 = recommend trimmin
9	Zero-Share Holdings	If 0 shares owned and not recommending BUY, use AVC

6.3 MEDIUM Mode System Instructions (5 rules)

Medium mode uses rules 5-9 from above (Cash Constraint, Tranching, Stop Loss, Profit-Taking, Zero-Share). Rules 1-4 are omitted because Bollinger-based mean-reversion and volume shock are short-term signals not relevant to a multi-week swing trading thesis. The SMA_50 and RSI_14 take their place as primary indicators.

6.4 AI Output Format

Both modes require the LLM to structure its response in exactly three sections:

- Section 1: A brief 1-2 sentence introduction summarizing the market environment.
- Section 2: A numbered list of priority actions (BUY, SELL, HOLD). AVOID and IGNORE are never listed here.
- Section 3: Stock-specific analysis for actioned stocks only. Maximum 3 sentences per stock. Indicators integrated naturally into prose (not as bullet points).

6.5 EOD Summary Prompt (4:05 PM Only)

`get_eod_summary_prompt()` uses a different instruction set that suppresses active trading advice. The AI is instructed to produce a 3-section response: (1) End-of-Day market summary, (2) Portfolio performance for currently held positions only, (3) 2-3 sentence outlook for tomorrow based on today's price action, volume, and breaking news.

6.6 Two-Step Intent Detection (Q&A Optimization)

When a user sends a free-form message to the Telegram bot (not "analyze" or a trade), the daemon runs a two-step process to avoid wasting tokens:

- Step 1 (Lightweight): A micro-prompt is sent to Gemini asking only whether the message is a "TRADE" or a "QUESTION". The prompt requests a JSON response. No dossier is included.
- Step 2A (TRADE path): If intent = TRADE, the extracted JSON (action, ticker, quantity, price) is parsed and staged in memory as a pending_trade, awaiting YES/NO confirmation.
- Step 2B (QUESTION path): If intent = QUESTION, the full dossier is generated, appended to a Q&A prompt, and sent to Gemini for a detailed response. This is the expensive path.

7. Telegram Daemon (telegram_daemon.py)

The Telegram daemon is a persistent Python process built on the python-telegram-bot library. It uses long-polling (not webhooks) to receive messages. It must be started manually and runs in the foreground.

7.1 Message Routing Logic

Incoming messages are routed through the following priority-ordered decision tree:

```
Incoming message
|
+-- Is chat_id authorized? (matches CHAT_ID in .env)
|   NO  -> Silently drop message (print to console only)
|   YES -> Continue
|
+-- Is there a pending_trade for this chat_id AND message is "yes"/"no"?
|   YES -> Confirm or cancel staged trade -> update_ledger() -> done
|
+-- Message is "change mode"?
|   YES -> Ask user to specify Short or Medium term mode
|
+-- Message is "short term mode" or "medium term mode"?
|   YES -> Write new mode to config.json -> done
|
+-- Message is "analyze" or "analysis"?
|   YES -> get_analysis_prompt() -> Gemini -> send_chunked_reply() -> log advice
|
+-- Everything else -> Two-step intent extraction
    TRADE intent  -> stage in pending_trades -> ask for confirmation
    QUESTION intent -> generate_dossier() -> Gemini Q&A -> reply
```

7.2 Trade Logging Flow

Trades go through a two-phase confirm-then-commit process to prevent accidental ledger writes:

- Phase 1 (Stage): Gemini extracts action, ticker, quantity, and price from the message. The trade dict is stored in the in-memory pending_trades{chat_id} dict.
- Phase 2 (Commit): User replies YES. The daemon calls update_ledger(action, ticker, qty, price). This function calls get_acb_and_balances() for current state, validates cash/share sufficiency, and appends a new row to trades.csv.
- Limitation: pending_trades is in-memory only. If the daemon restarts between Phase 1 and Phase 2, the staged trade is lost and must be re-entered.

7.3 update_ledger() Function

This function writes new trade rows to trades.csv. It validates before writing:

- BUY validation: if cash_bal < cost, raises ValueError("Insufficient cash!")
- SELL validation: if current_shares < quantity, raises ValueError("Insufficient shares")
- Appends a CSV row: [Date, Ticker, Action, Quantity, Price, Shares_Balance, Cash_Balance]

7.4 Market Hours Warning

When the user sends "analyze" outside of trading hours (9:30 AM to 4:00 PM ET, Mon-Fri), the response is prefixed with a warning: "The market is currently closed. This analysis is based on the last session's closing data." Note: the time check uses local system time and assumes the machine is in the ET timezone.

7.5 Message Chunking

Telegram enforces a hard 4096-character limit per message. The `send_chunked_reply()` async function splits any response into 4000-character chunks and sends them sequentially to avoid truncation.

8. Scheduled Cron Jobs & Automated Tasks

The three scheduled tasks are managed by the Antigravity AI agent using its native schedule tool. Each task fires a cron-based wake-up, executes a Python one-liner to generate the appropriate prompt, calls the Gemini API to generate a response, and sends it via `send_telegram.py`.

8.1 Task 1: Hourly Market Scan

Parameter	Value
Cron Expression	0 10-15 * * 1-5
Schedule	Every hour on the hour, 10 AM to 3 PM ET, Monday to Friday
Prompt Source	<code>get_analysis_prompt()</code> from <code>market_analyzer.py</code>
Output	SHORT or MEDIUM mode trading recommendation sent via Telegram

8.2 Task 2: Pre-Close Market Scan (3:30 PM)

Parameter	Value
Cron Expression	30 15 * * 1-5
Schedule	Every weekday at 3:30 PM ET
Prompt Source	<code>get_analysis_prompt()</code> from <code>market_analyzer.py</code>
Output	Final intraday recommendation before market close. The last advice logged to <code>advice_history.txt</code> for the day

8.3 Task 3: End-of-Day Summary (4:05 PM)

Parameter	Value
Cron Expression	5 16 * * 1-5
Schedule	Every weekday at 4:05 PM ET (5 minutes after market close)
Prompt Source	<code>get_eod_summary_prompt()</code> from <code>market_analyzer.py</code>
Output	Holistic EOD summary: portfolio performance for the day and next-day outlook.

8.4 Advice History: Intraday vs End-of-Day Retention

Throughout the day, every recommendation generated by the cron tasks or `/analyze` command is appended to `advice_history.txt`. When `get_cleaned_advice_history()` runs (on every dossier generation), it applies the following retention policy:

- Today's entries: ALL entries are retained to give the AI full intraday context.
- Previous days: ONLY the last entry (typically the 3:30 PM advice) is retained per day.
- History limit: Maximum 5 previous business days. Older entries are permanently deleted.

9. Trade Ledger (trades.csv)

trades.csv is the single source of truth for all financial positions. It is an append-only CSV file. The daemon appends new rows via `update_ledger()`. Manual edits are possible but must preserve the column schema exactly.

9.1 Schema

Column	Type	Description
Date	YYYY-MM-DD HH:MM:SS	Timestamp of the trade (local system time).
Ticker	String	TSX ticker symbol, e.g., "CNQ.TO". Always uppercase.
Action	String	"BUY", "SELL", or "INIT" (initial portfolio seeding). INIT is used to seed the initial cash balance.
Quantity	Float	Number of shares traded.
Price	Float	Per-share price at time of trade.
Shares_Balance	Float	Running shares balance for this ticker after the trade. Should be 0 for INIT.
Cash_Balance	Float	Total available free cash after the trade. This is the canonical cash figure.

9.2 Example

```
Date,Ticker,Action,Quantity,Price,Shares_Balance,Cash_Balance
2026-06-14 09:00:00,INIT,INIT,0,0,0,20000.00
2026-06-14 10:30:00,CNQ.TO,BUY,100,62.50,100.0,13750.00
2026-06-15 14:00:00,CNQ.TO,BUY,50,60.12,150.0,10756.00
2026-06-16 11:00:00,CNQ.TO,SELL,30,64.00,120.0,12676.00
```

9.3 Important Notes

- INIT action: The very first row is typically an INIT action used to seed the initial cash balance. The Ticker column value for INIT rows is ignored by the ACB calculation.
- Cash_Balance is the canonical cash figure: `get_acb_and_balances()` reads the last Cash_Balance value from the file. If this column is ever manually set incorrectly, the cash constraint logic will be wrong.
- The file uses UTF-8 encoding and should use consistent line endings (LF or CRLF). Python's csv module with `newline=""` handles this transparently on append.

10. Advice History System (advice_history.txt)

advice_history.txt provides the AI with rolling memory of its own recommendations. Without it, the AI would have no context about what it said earlier in the day and could give contradictory advice. The file uses a simple timestamp-keyed format.

10.1 File Format

```
[2026-06-16 10:00:15]
The TSX market opened with moderate volatility...
1. HOLD CNQ.TO

[2026-06-16 11:00:22]
Oil prices have recovered slightly...
1. HOLD CNQ.TO

[2026-06-15 15:30:07]
(Final advice from previous business day)
1. BUY CNQ.TO
```

10.2 Writers

- send_telegram.py: Appends the full response text as one entry after all chunks are sent. Used by the scheduled cron tasks.
- telegram_daemon.py: Appends the full response text directly after the Gemini API call when handling an /analyze command.

10.3 Pruning (get_cleaned_advice_history)

On every call, the function: (1) Parses all timestamp blocks via regex. (2) Groups entries by calendar date. (3) Keeps ALL entries for today. (4) Keeps ONLY the last entry per day for the last 5 business days (Mon-Fri). (5) Deletes all other entries. (6) Rewrites the file with the cleaned set. (7) Returns the cleaned history as a string for dossier injection.

10.4 Concurrency Warning

Both send_telegram.py and get_cleaned_advice_history() write to this file. There is no file locking mechanism. In practice, the cron agent runs synchronously (one script at a time) and the daemon only writes after Gemini responds, making simultaneous writes unlikely. However, if a cron task fires exactly while the daemon is generating a dossier, a small risk of data loss exists. A future improvement would be to use OS-level file locking.

11. Dependency Reference (requirements.txt)

Package	Pinned Version	Purpose
yfinance	1.4.1	Fetches OHLCV price history and ticker metadata (ex-di
pandas	3.0.1	DataFrame manipulation for price history and indicator c
pandas-ta	0.4.71b0	Technical Analysis library. Computes EMA, RSI, ATR, S
python-telegram-bot	22.8	Async Telegram Bot API wrapper. Powers the daemon's
google-genai	2.8.0	Google Gemini API client. Used to call gemini-2.5-flash f
python-dotenv	1.2.2	Reads the .env file and injects variables into os.environ.
requests	latest	HTTP client. Used by send_telegram.py to call the Telegr
fpdf2	2.8.7	PDF generation library. Used exclusively by generate_po

11.1 Installation

```
# Install all dependencies:  
C:\Users\Aamir\miniforge3\python.exe -m pip install -r requirements.txt
```

12. Security Hardening

12.1 Telegram Bot Authorization

The daemon compares every incoming message's `chat_id` against the `CHAT_ID` value in `.env`. If they do not match, the message is silently dropped (no reply) and a warning is logged to the console. This prevents unauthorized users from logging trades or extracting portfolio data even if they discover the bot token.

```
# In handle_message():
if str(chat_id) != os.getenv("CHAT_ID"):
    print(f"Unauthorized access attempt from {chat_id}")
    return
```

12.2 Secrets Management

- `.env` is excluded from version control by `.gitignore`.
- `trades.csv` and `advice_history.txt` are also excluded, as they contain sensitive financial data.
- No credentials are hardcoded in any `.py` file. All secrets are read from `os.getenv()`.

12.3 Input Validation

- Trade quantity and price are parsed by the Gemini model and then cast to `float()` with explicit error handling. A `ValueError` terminates processing safely.
- Ticker validation: The intent-extraction prompt is given the list of valid tickers from `config.json`, instructing the model to only extract trades for configured symbols.
- Cash and share sufficiency checks in `update_ledger()` prevent the ledger from entering a negative balance state.

13. Operational Runbook

13.1 Daily Startup

Step 1: Open File Explorer and navigate to c:\Users\Aamir\OneDrive\TSX-Trader
 Step 2: Double-click start_daemon.bat
 (This runs: C:\Users\Aamir\miniforge3\python.exe telegram_daemon.py)
 Step 3: Verify the console shows: "Daemon listening for Telegram messages..."
 Step 4: The Antigravity Agent will automatically send its first
 analysis at 10:00 AM ET via the cron schedule.

13.2 Logging a Trade via Telegram

You: I bought 50 CNQ at 60.50
 Bot: Did you mean to BUY 50.0 shares of CNQ.TO @ \$60.50?
 Reply YES to confirm or NO to cancel.
 You: yes
 Bot: Confirmed and Recorded: BUY 50.0 CNQ.TO @ \$60.50
 CNQ.TO: 150.0 shares | Cash: \$10,681.00

13.3 Requesting On-Demand Analysis

You: analyze
 Bot: Generating on-demand analysis. Please wait...
 [Returns full SHORT or MEDIUM mode recommendation]

13.4 Asking a Market Question

You: What is the current RSI on CNQ?
 Bot: Reading live market data to answer your question...
 [Returns dossier-backed answer from Gemini]

13.5 Switching Trading Mode

You: change mode
 Bot: Please reply with either Short Term Mode or Medium Term Mode.
 You: Medium Term Mode
 Bot: Operating mode successfully updated to: MEDIUM TERM.

13.6 Restoring the Agent After Session Loss

If the Antigravity agent session is lost (e.g., browser closed, session expired), the AGENT_RESTORE_INSTRUCTIONS.md file in the project root contains the exact cron expressions and prompt text needed to recreate all three scheduled tasks. Open a new session, share the file with the agent, and follow its instructions.

14. Troubleshooting Guide

14.1 Common Issues

Bot is not responding to Telegram messages

Cause 1: telegram_daemon.py is not running. Check for the console window or re-run start_daemon.bat.

Cause 2: TELEGRAM_BOT_TOKEN or CHAT_ID in .env is wrong. Verify with BotFather and your Telegram account.

Cause 3: chat_id mismatch. Check the console for "Unauthorized access attempt" log lines.

Analysis shows "Error fetching [TICKER]"

Cause: Yahoo Finance is temporarily unavailable or the ticker symbol is invalid. Wait a few minutes and retry, or verify the ticker in config.json is a valid Yahoo Finance symbol.

Cron-scheduled messages have stopped arriving

Cause: The Antigravity agent session has been lost or the schedule tasks were cancelled. Open a new session and use AGENT_RESTORE_INSTRUCTIONS.md to recreate the three cron tasks.

"Insufficient cash" error when logging a BUY

Cause: The Cash_Balance in trades.csv shows insufficient funds for the trade. Check the most recent Cash_Balance value in trades.csv. If incorrect due to a manual edit, fix the last row's Cash_Balance column.

ValueError: Invalid action when logging a trade

Cause: The Gemini intent-extraction returned an unexpected action value. This can happen if the user's message is ambiguous. Rephrase the trade clearly: "I bought 100 CNQ at 60.12".

Telegram messages are being truncated

Cause: A very long analysis is being sent by a code path that does not use chunking. The daemon's send_chunked_reply() handles this for /analyze and Q&A. The CLI send_telegram.py also chunks to 4000 characters. If truncation occurs, check that the calling code is routing through one of these handlers.

Advice history is empty on first run of the day

This is expected behaviour. advice_history.txt is populated throughout the day as scheduled and on-demand analyses are sent. By the second hourly analysis, the AI will have its first advice entry available as context.